Name: Sanjog Poojary

Roll no : 38

Experiment 7

Aim

Experiment to implement any one of the clustering algorithms using Rapid Miner and Python.

Theory

- Discover the natural grouping or structure in unlabelled data without predefined categories.
- Data points are assigned to clusters based on similarity or distance measures.
- Uses Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.

Types of Clustering

- Hard Clustering Hard clustering assigns each data point to exactly one cluster. A data point cannot belong to multiple clusters, making the grouping clear and easy to interpret.
- Each data point belongs to only one cluster
- No overlap between clusters
- Simple and easy to interpret
- Soft Clustering Soft clustering allows a data point to belong to multiple clusters with different probabilities. Instead of assigning a strict cluster, it gives a degree of membership to each cluster. Example A data point may belong 70% to Cluster 1 and 30% to Cluster 2, indicating that it shares characteristics with both groups.

Use Cases

- Overlapping class boundaries: Useful when data points cannot be clearly separated into distinct groups.
- Customer personas: Helps represent customers who share traits with multiple behavioral groups.
- Medical diagnosis: Patients may show symptoms related to multiple conditions. Benefits
- Captures ambiguity: Represents uncertainty when cluster boundaries are not clear.
- Models gradual transitions: Allows smooth transitions between clusters instead of strict separation.

K-Means Clustering: K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data. Uses of K-Means Clustering K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

- Data Segmentation: One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
- Image Compression: K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
- Anomaly Detection: K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
- Document Clustering: In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
- Organizing Large Datasets: When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

Dataset Overview

- **Dataset:** JioHotstar Preprocessed Dataset (33,171 records, 23 regular attributes).
- **Tool:** Altair AI Studio 2026.1 (RapidMiner), Free licence.
- **Licence limit:** the Free licence processes 10,000 examples, so the process runs on 10,000 of the 33,171 records.

Implementation (RapidMiner)

Process built in the design view: Retrieve → Select Attributes → Replace Missing Values → Normalize → Clustering (K-Means, 4 clusters).

- **Retrieve:** loads the JioHotstar dataset from the local repository.
- **Select Attributes:** keeps the numerical viewing features.
- **Replace Missing Values:** replaces missing numerical values with the attribute average.
- **Normalize:** Z-transformation of the selected numerical features.
- **Clustering (K-Means):** k = 4 clusters.

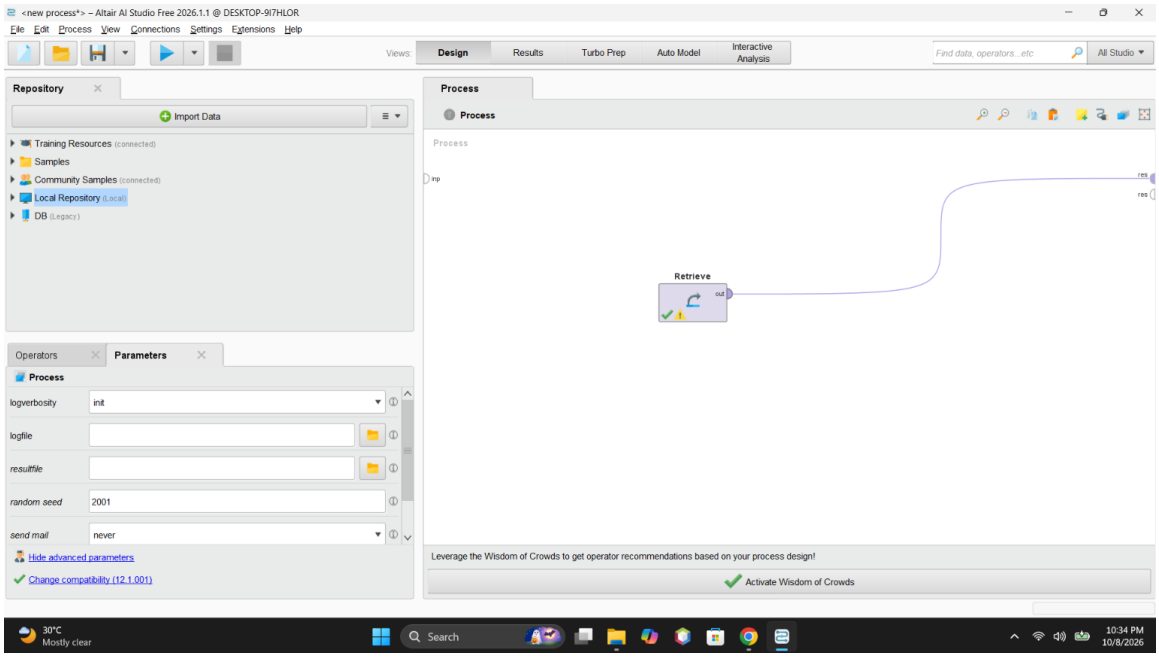


Figure 1: Retrieve operator added to the process

Inference: The Retrieve operator was added to the process to access the JioHotstar dataset stored in the repository. This operator loads the saved dataset into Altair AI Studio so that it can be processed and analyzed using subsequent operators.

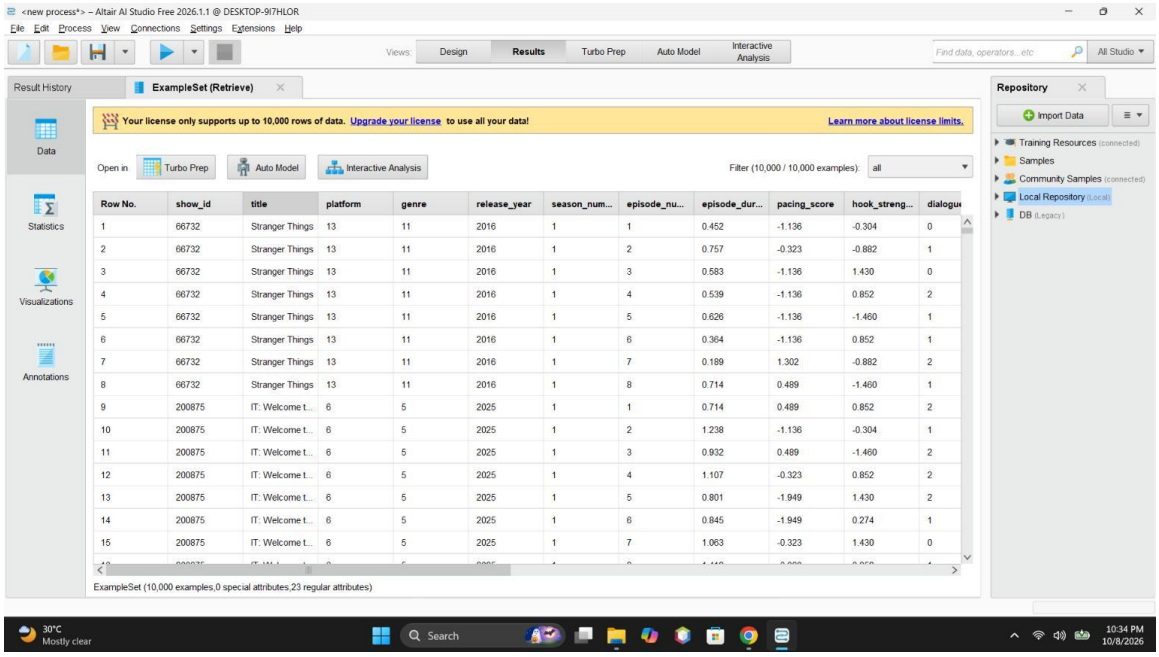


Figure 2: Retrieved dataset in the Results view (10,000 of 33,171 examples under the Free licence)

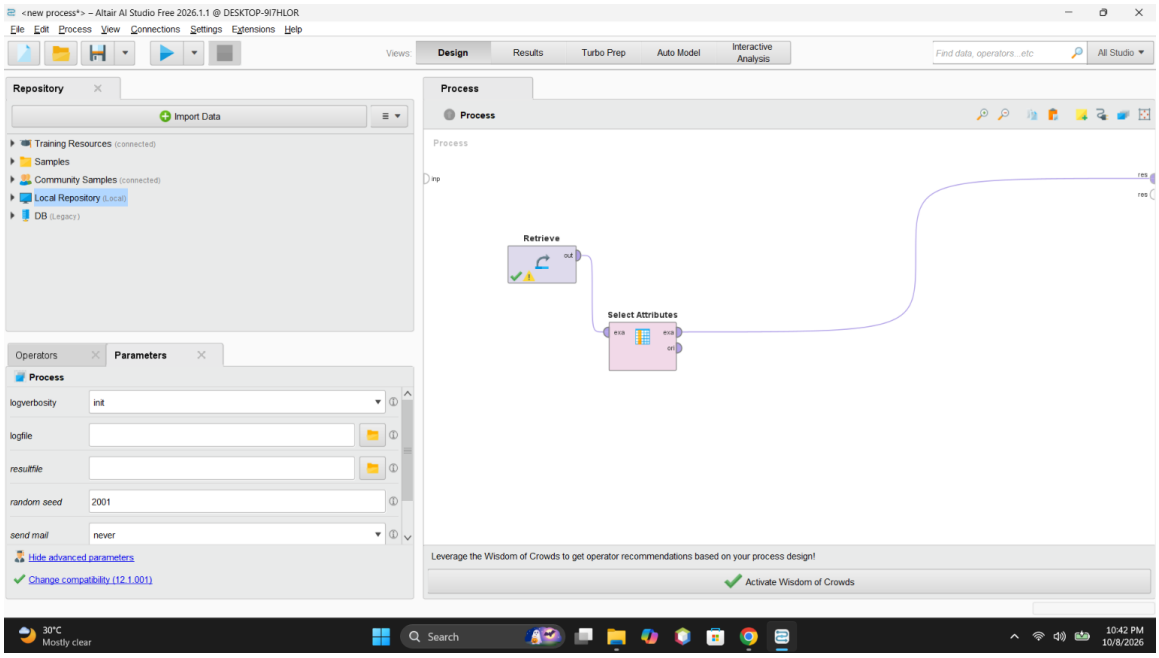


Figure 3: Select Attributes added after Retrieve

Inference: The dataset was successfully retrieved and displayed in the Results view. The Free license restricted processing to 10,000 examples, and the displayed dataset contains 10,000 rows and 23 regular attributes. The data includes show details and numerical viewing-related features required for further analysis.

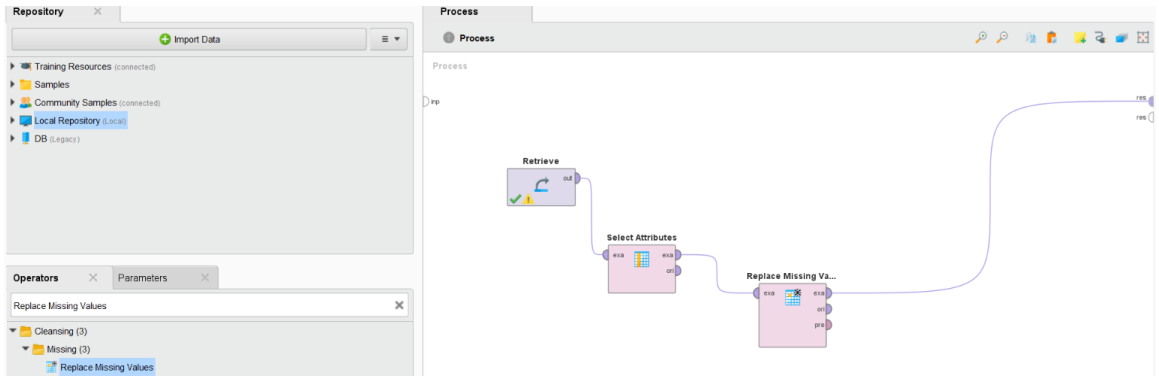


Figure 4: Replace Missing Values added after Select Attributes

Inference: The Replace Missing Values operator was added after Select Attributes. It is used to handle missing data by replacing missing numerical values with the average of the respective attributes. This improves data completeness before normalization and clustering.

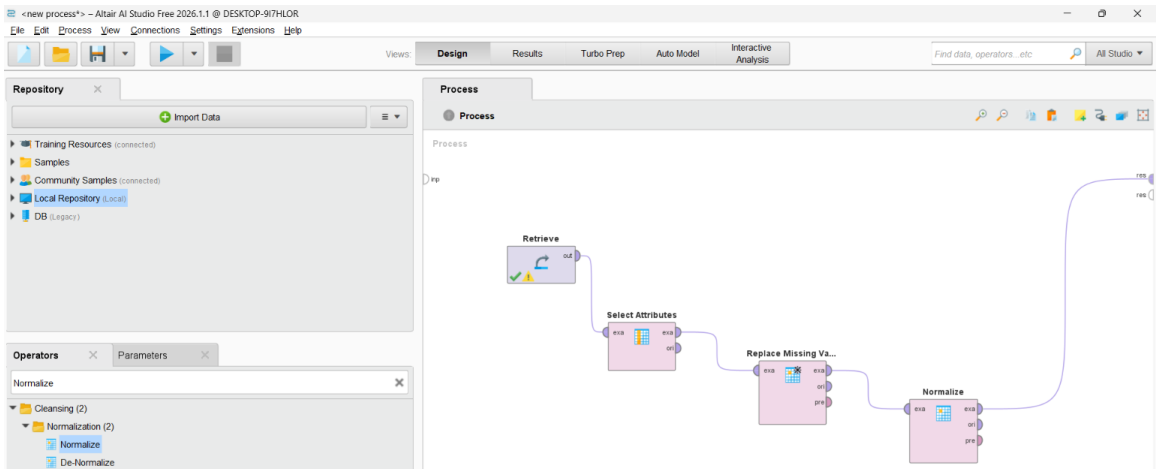


Figure 5: Normalize operator connected after Replace Missing Values

Inference: The Normalize operator was connected after Replace Missing Values to standardize the selected numerical features using Z-transformation. This makes the features comparable in scale and prevents attributes with larger numerical ranges from dominating the distance calculations performed by K-Means.

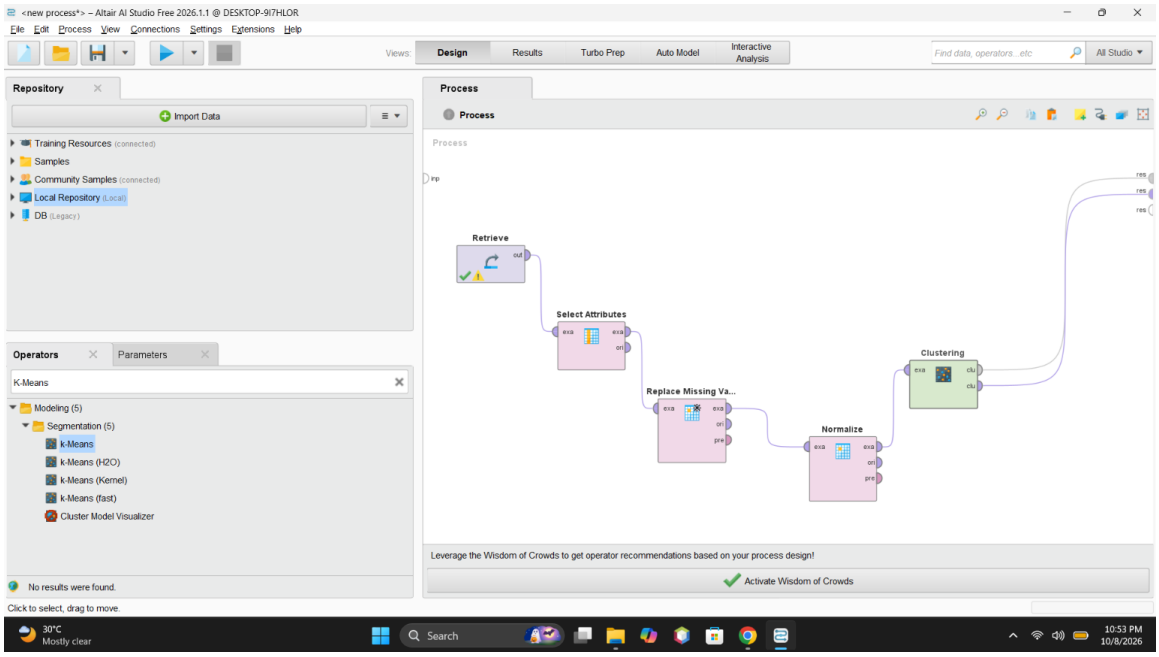


Figure 6: K-Means (Clustering) operator added to the complete workflow

Inference: The K-Means clustering operator was added to the workflow after data preparation. The process connects Retrieve, Select Attributes, Replace Missing Values, Normalize, and Clustering in sequence. The prepared dataset is passed to K-Means to group similar records into four clusters and analyze different viewing-behaviour patterns.

After executing the process, Altair AI Studio displays the resulting ExampleSet containing the assigned cluster for each record. The cluster column indicates which cluster each record belongs to, such as cluster_0 or cluster_2. The numerical attribute values are standardized values because the Normalize

operator uses Z-transformation. The results show that the records have been grouped according to similarities in their selected attributes.

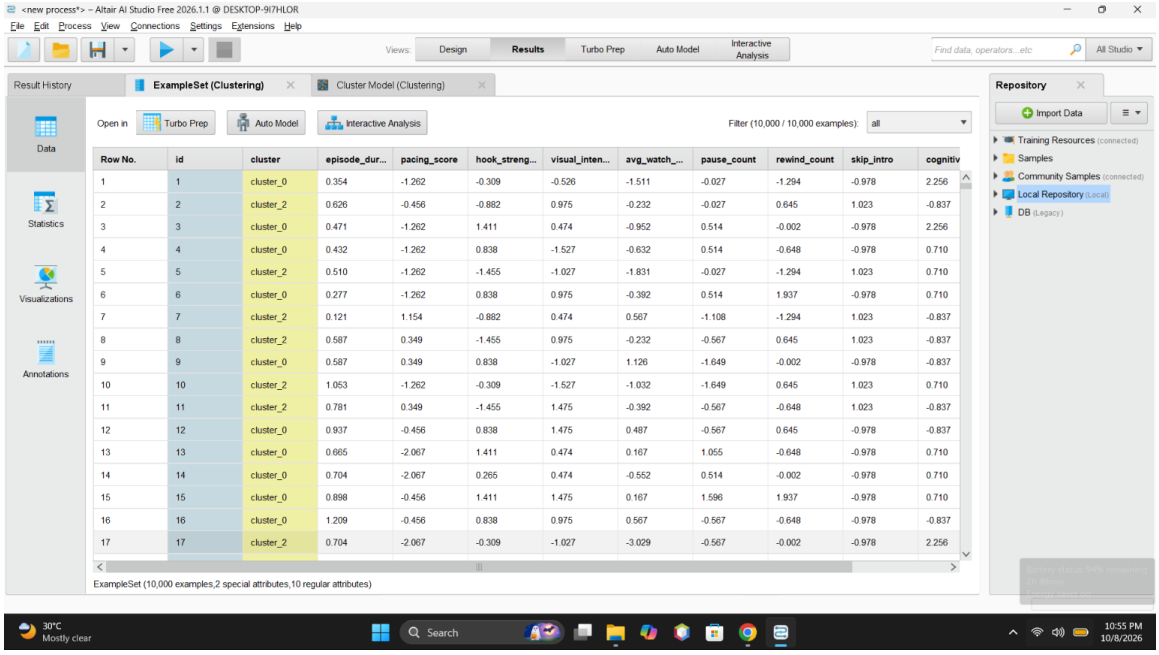


Figure 7: ExampleSet with the assigned cluster for each record

This screenshot shows another portion of the ExampleSet results table. Each row represents a record from the dataset, and the cluster column shows its assigned group. Viewing multiple rows helps verify that the clustering operator has assigned cluster labels throughout the processed dataset. The displayed values are normalized values rather than the original measurements.

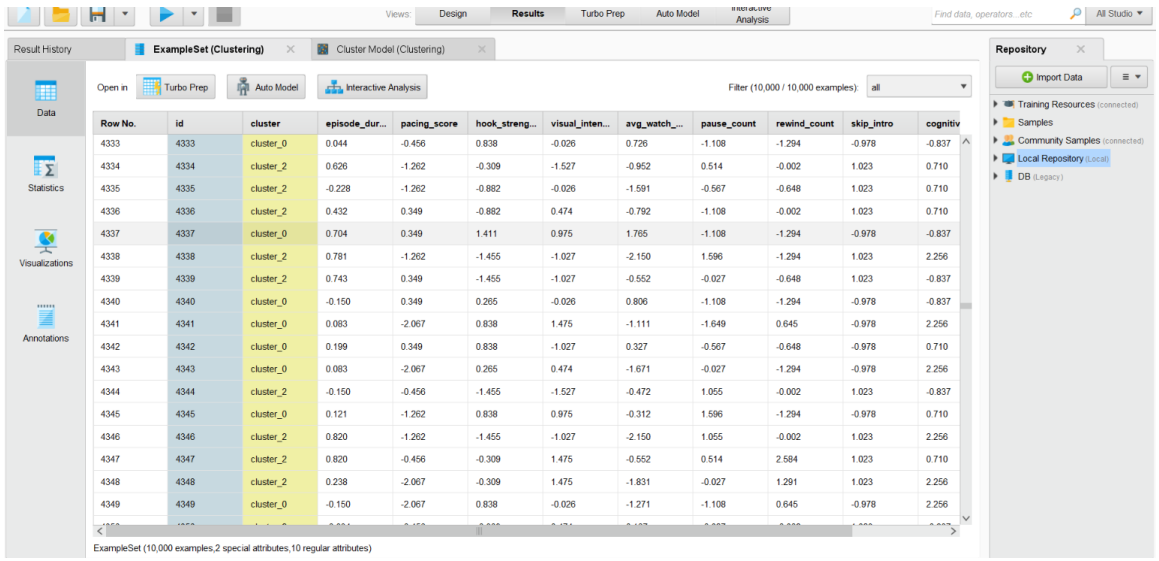


Figure 8: Further rows of the clustered ExampleSet

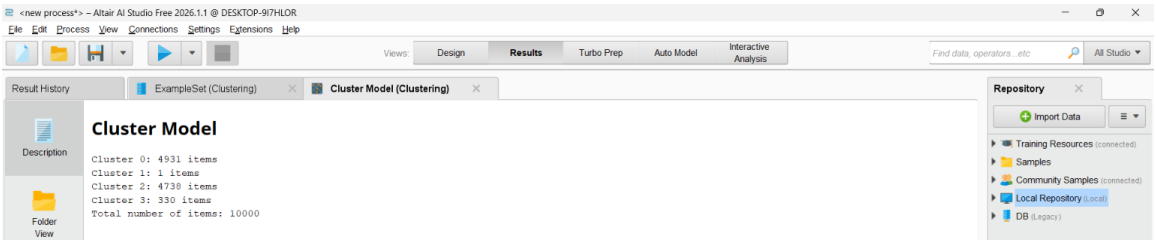


Figure 9: Cluster Model: number of items in each cluster

The Cluster Model displays the number of records assigned to each cluster. The output shows that Cluster 0 contains 4,931 records, Cluster 1 contains 1 record, Cluster 2 contains 4,738 records, and Cluster 3 contains 330 records. The total number of processed records is 10,000. The results indicate that most records belong to Clusters 0 and 2, while Cluster 1 contains only one record, which may represent an unusual observation or outlier.

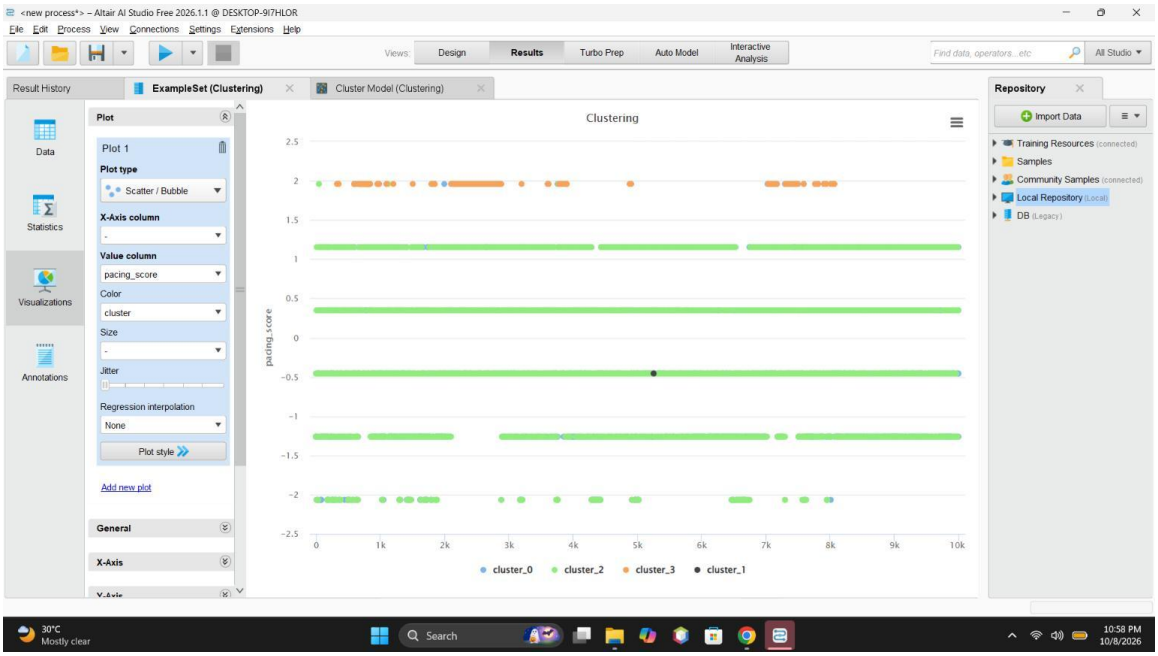


Figure 10: Scatter plot of pacing_score coloured by cluster

The visualization displays the clustering results using different colours to represent the clusters. Each plotted point represents a record, and the colours distinguish the cluster assignments. This visualization helps observe how records are distributed among the identified groups. The displayed graph uses pacing_score as the value column and cluster for colour. Since the X-axis is not assigned to a specific dataset attribute, this graph is mainly useful for viewing the distribution of cluster labels against the selected values.

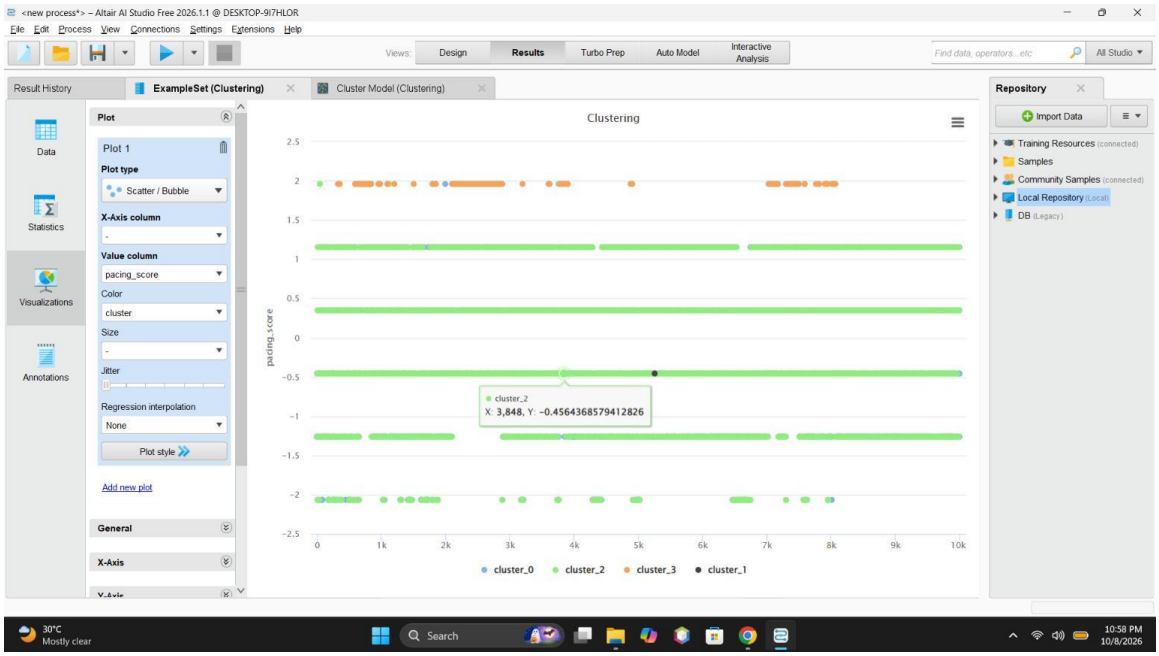


Figure 11: Scatter plot with tooltip for a single record (cluster_2)

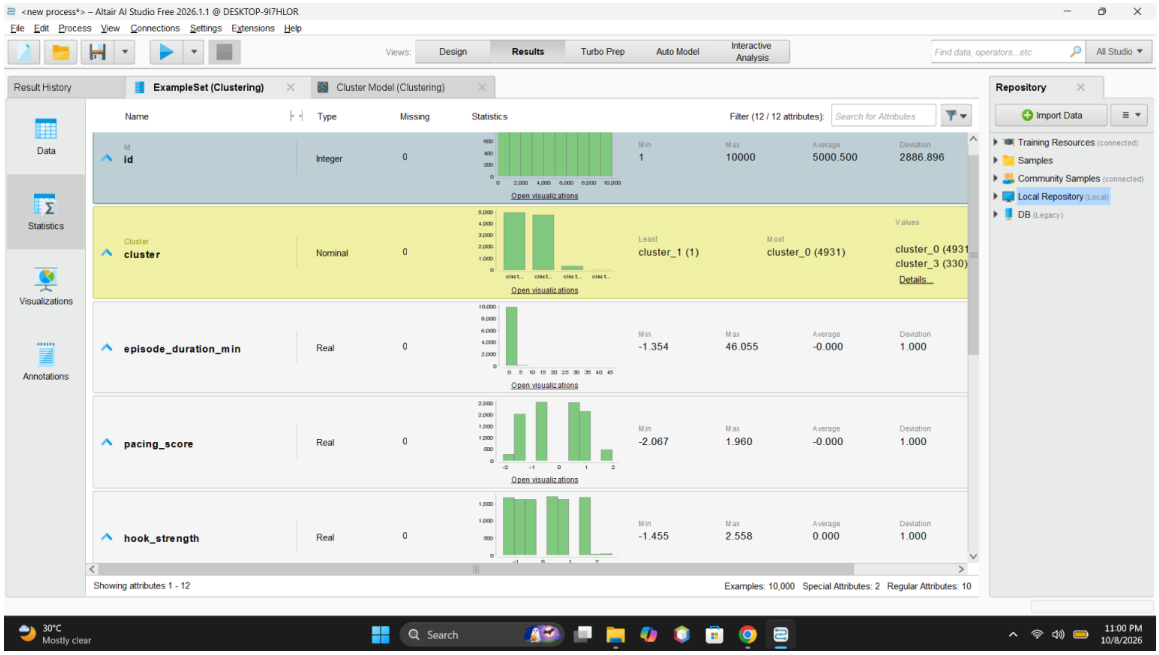


Figure 12: Statistics view of the clustered attributes

The Statistics view displays information about the attributes in the clustered dataset, such as their data types, missing-value counts, minimum values, maximum values, averages, and deviations. The screenshot shows the distribution of the cluster attribute and statistics for numerical attributes such as episode duration, pacing score, and hook strength. This helps verify the processed data and understand its distribution. The normalized numerical attributes have averages close to zero and standard deviations close to one, as expected from Z-transformation.

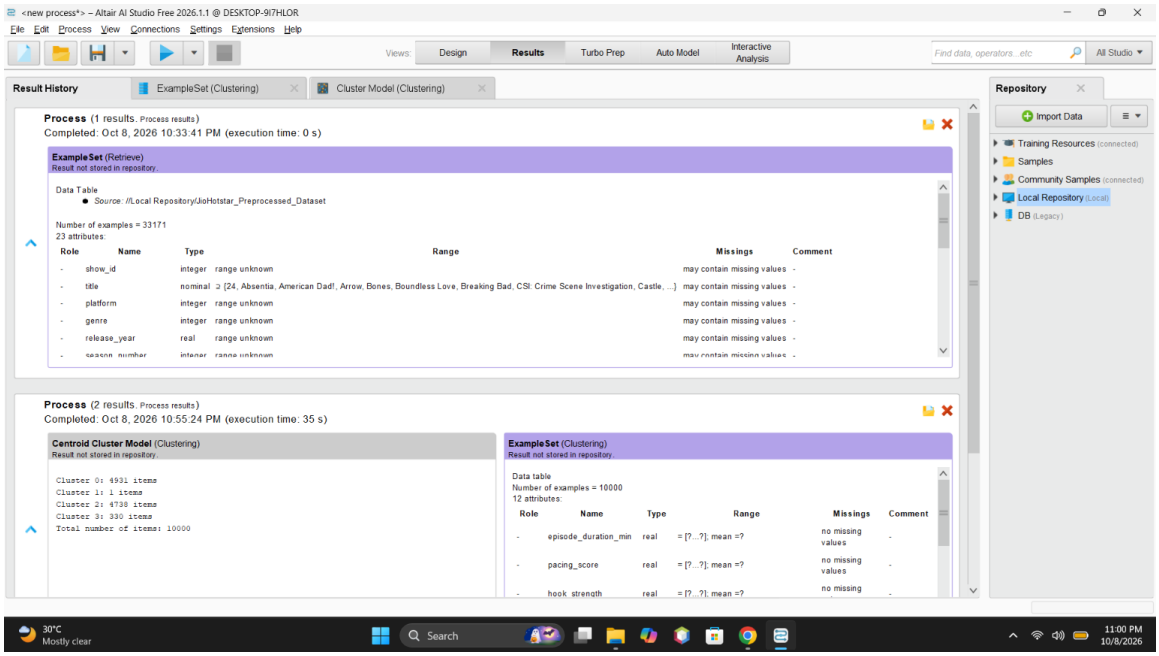


Figure 13: Result History showing the retrieved and clustered data

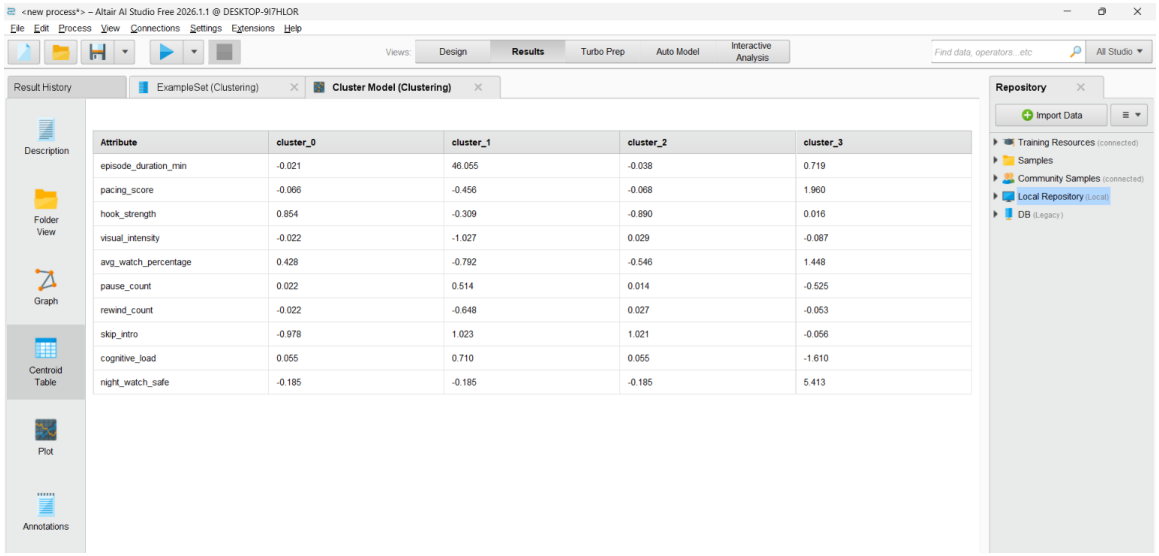


Figure 14: Centroid table: attribute means of each cluster

The Result History displays the outputs generated after executing the process. It confirms that the original dataset was retrieved with 33,171 records and that the clustering output contains 10,000 records. The Cluster Model output also displays the number of records assigned to each cluster. The reduction to 10,000 records was due to the example limit in the free version of Altair AI Studio. This screenshot provides an overall confirmation of the execution and the generated results.

Results

Table 1: Cluster sizes from the RapidMiner Cluster Model

Cluster	Records (of 10,000)

Cluster 0	4,931
Cluster 1	1
Cluster 2	4,738
Cluster 3	330

Comparison

Table 2: Comparison of the Python and RapidMiner runs

Metric / aspect	Experiment 6: Python (scikit-learn K-Means)	Experiment 7: Altair AI Studio (K-Means)
Number of clusters (k)	4	4
Dataset records processed	33,171	10,000
Cluster counts	13,101; 10,555; 8,988; 527	4,931; 1; 4,738; 330
Silhouette score	0.2084	Not provided
Inertia	195,849.28	Not provided
Davies-Bouldin index	1.3747	Not provided
Preprocessing	StandardScaler	Replace Missing Values and Normalize (Z-transformation)
Visualization	Scatter plots with matplotlib	Scatter/bubble plot coloured by cluster
Main observation	Four viewing-behaviour groups	Most records in Clusters 0 and 2; Cluster 1 has one record

Table 3: Python versus RapidMiner

Feature	Python	RapidMiner
Customisation	High flexibility	Low (GUI based, pre-set options)
Ease of use	Requires coding knowledge	Easier for beginners
Evaluation metrics	Silhouette score used for validation	No explicit evaluation metric
Visualisation	Customisable with Matplotlib/Seaborn	Built-in visualisation tools

Conclusion

Thus, K-Means clustering was successfully performed on the JioHotstar dataset using Altair AI Studio. The dataset was preprocessed by handling missing values and applying Z-transformation normalization before grouping the records into four clusters. Out of 10,000 processed records, most belonged to Clusters 0 and 2. The experiment helped me understand how clustering can group similar episodes based on viewing and engagement characteristics. One limitation was the free-version restriction of 10,000 records.

GitHub Folder: https://github.com/Sanjog-Poojary/DMBI-Lab/tree/main/Exp7_KMeans_RapidMiner